



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

Отчёт по лабораторной работе №4 по дисциплине «Анализ алгоритмов»

Тема Программирование параллельных потоков

Студент Попов Ю.А.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Трассировка лучей	5
1.2 Параллельное исполнение программы	5
1.2.1 Зелёные потоки	5
1.2.2 Многопроцессная обработка	6
1.2.3 Нативные потоки	6
1.2.4 Выбор метода распараллеливания программы	7
2 Конструкторская часть	8
2.1 Разработка алгоритмов	8
2.2 Разработка средства для параллельного выполнения программы	11
2.3 Генерация изображения по областям	12
2.4 Типы данных	13
3 Технологическая часть	14
3.1 Средства реализации	14
3.1.1 Реализация пула потоков	14
3.2 Управление нативными потоками	15
3.3 Реализация замеров времени выполнения реализации алгоритмов	16
3.4 Реализация генерации изображения по областям	16
3.5 Реализация алгоритмов	17
3.6 Тестирование ПО	19
4 Исследовательская часть	20
4.1 Технические характеристики	20
4.2 Временя выполнения последовательной и параллельной реализации алгоритма	20
4.3 Исследование зависимости времени выполнения реализации алгоритма от числа потоков	21

ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ВВЕДЕНИЕ

Существуют программы, в которых требуется обрабатывать большие объёмы данных или производить множество операций, зависящих от процессора. Пример такой программы — визуализация сцены с использованием алгоритма трассировки лучей, в котором обрабатываются несколько десятков миллионов операций. Для того чтобы ускорить время выполнения программы, используется **параллельное исполнение алгоритмов**.

Параллельное исполнение — способ, при котором происходит одновременное исполнение алгоритма в нескольких потоках.

Целью данной работы является разработать и провести сравнительный анализ последовательного и параллельного алгоритма трассировки лучей, используемого для генерации изображения. Для достижения цели поставлены следующие задачи:

- описать последовательный алгоритм решения задачи;
- разработать параллельную версию алгоритма;
- реализовать обе версии алгоритма;
- выполнить сравнительный анализ зависимостей времени решения задач от размерности входа для реализации последовательного алгоритма и для реализации модифицированного алгоритма;
- выполнить сравнительный анализ зависимостей времени решения задач от размерности входа для реализации модифицированного алгоритма при $k \in \{1, 2, 4, 8, \dots, 8 \cdot q\}$, где q — количество логических ядер процессора;
- сформулировать рекомендацию о выборе k для решения задачи на ЭВМ выбранной автором работы архитектуры.

1 Аналитическая часть

В данной части будут рассмотрены последовательный и параллельный варианты алгоритма генерации изображения на основе трассировки лучей. Также будут представлены возможные подходы к распараллеливанию программы и осуществлён выбор одного из них.

1.1 Трассировка лучей

Трассировка лучей математически моделирует поведение света и его взаимодействие с физическими объектами — отражение, преломление и тени. Сложные методы трассировки лучей позволяют создавать реалистичные спецэффекты для фильмов и игр [1].

Из каждого пикселя камеры испускается луч. При попадании этого луча на какую либо поверхность, он отражается с учётом материала поверхности. Данный процесс может быть рекурсивным, что позволяет визуализировать переотражения любой сложности.

Алгоритм трассировки лучей выполняется для каждого пикселя сцены. Из-за присутствия материалов с шершавой поверхностью возникает элемент случайности, в результате которого картинка становится шумной. Чтобы избавиться от этого недостатка необходимо запускать луч несколько раз, и усреднять полученные результаты. К тому же сам алгоритм является достаточно трудоёмким с точки зрения временной эффективности.

Однако, алгоритм подходит для распараллеливания, так как изображение можно разделить на отдельные участки, например на области или на сканирующие строки. Участки не зависят друг от друга, так как в расчёте не происходит изменения данных, которые читают разные потоки. Таким образом отпадает необходимость использовать методы синхронизации потоков.

1.2 Параллельное исполнение программы

Параллельное исполнение программы или её частей можно организовать несколькими способами:

- зелёные потоки.
- нативные потоки;
- мультипроцессинг.

1.2.1 Зелёные потоки

Зелёные потоки — это легковесные потоки выполнения, создаваемые и управляемые в пользовательском пространстве средой выполнения (например, виртуальной машиной или библиотекой), а не ядром операционной системы. В отличие от нативных потоков, планирование и переключение между зелёными потоками осуществляется без прямого участия ОС.

Зелёные потоки эффективнее при создании и переключении, эта особенность делает их полезными в системах с высокой конкуренцией задач. Однако их ключевое ограничение

заключается в том, что они не могут использовать несколько ядер процессора для истинного параллелизма: поскольку операционная система видит только один нативный поток, все зелёные потоки выполняются конкурентно, но не параллельно. Кроме того, блокирующая операция в одном зелёном потоке может остановить выполнение всех остальных.

1.2.2 Многопроцессная обработка

Многопроцессная обработка — это модель выполнения программ, при которой вычислительная нагрузка распределяется между несколькими независимыми процессами, каждый из которых обладает собственным адресным пространством и ресурсами. В отличие от многопоточности, где потоки разделяют память внутри одного процесса, процессы в многопроцессной обработке изолированы друг от друга, а их взаимодействие осуществляется с помощью механизмов межпроцессного взаимодействия, таких как: каналы, очереди сообщений, сокеты или разделяемая память. Эта модель эффективна на многоядерных процессорах, так как операционная система может распределять процессы по разным ядрам центрального процессора. Основные преимущества многопроцессной обработки:

- возможность полного использования аппаратного параллелизма;
- высокая отказоустойчивость (сбой в одном процессе, не затрагивает другие);
- отсутствует необходимость синхронизировать доступ к памяти.

В то же время мультипроцессинг имеет и существенные недостатки:

- создание и переключение между процессами сопряжено со значительными накладными расходами по сравнению с потоками;
- обмен данными между процессами требует явного использования IPC-механизмов, что усложняет разработку и снижает производительность;
- каждый процесс потребляет дополнительную память, так как содержит собственную копию исполняемого кода и данных.

1.2.3 Нативные потоки

Поток — это базовая единица выполнения в рамках процесса, представляющая собой последовательность инструкций, которая может выполняться независимо от других потоков того же процесса. Все потоки, принадлежащие одному процессу, разделяют его адресное пространство, включая код программы, глобальные переменные и открытые ресурсы, но при этом обладают собственными стеками и состояниями выполнения. Такая модель позволяет эффективно реализовывать конкурентность. На многоядерных системах потоки могут распределяться по разным ядрам, обеспечивая истинный параллелизм.

Основное преимущество потоков — низкие накладные расходы по сравнению с процессами. Создание, переключение и взаимодействие между потоками происходит значительно быстрее, поскольку они используют общую память и не требуют сложных механизмов межпроцессного взаимодействия. Это делает многопоточность особенно эффективной для задач,

требующих интенсивного обмена данными. Однако именно разделяемая память порождает ключевые проблемы многопоточного программирования: состояния гонки и взаимные блокировки, возникающие при некорректном доступе к общим данным. Для их предотвращения необходимо использовать примитивы синхронизации, такие как: мьютексы, семафоры, условные переменные. Это усложняет код и снижает его производительность.

1.2.4 Выбор метода распараллеливания программы

Для распараллеливания программы в лабораторной работе были выбраны нативные потоки, так как мультипроцессинг плохо подходит ввиду сложности организации доступа к разделяемой памяти, а использование зелёных потоков запрещено по условию работы.

Вывод

В данной части были представлены последовательный и параллельный варианты алгоритма трассировки лучей, а также выбран подход к организации параллельного выполнения программы.

2 Конструкторская часть

В данной части будут представлены схемы последовательного и параллельного алгоритма, реализации функции генерации участка изображения и реализации рабочего потока.

2.1 Разработка алгоритмов

Для обоих алгоритмов использовалась одна и та же функция для генерации участка изображения. В последовательном алгоритме она вызывается один раз для всего изображения, а в параллельном для каждой области.

На рисунке 2.1 изображена схема реализации последовательного алгоритма.

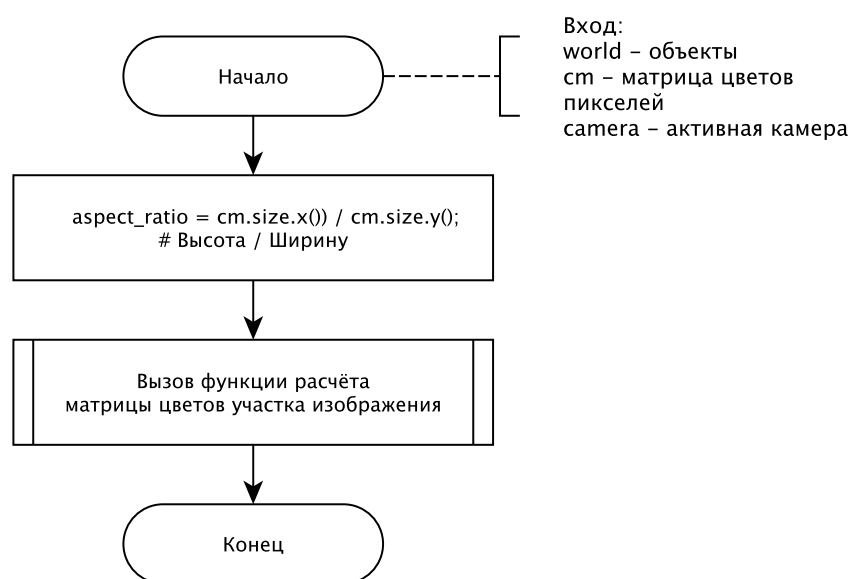


Рисунок 2.1 — Схема реализации последовательного алгоритма

На рисунке 2.2 изображена схема реализации алгоритма генерации участка изображения.

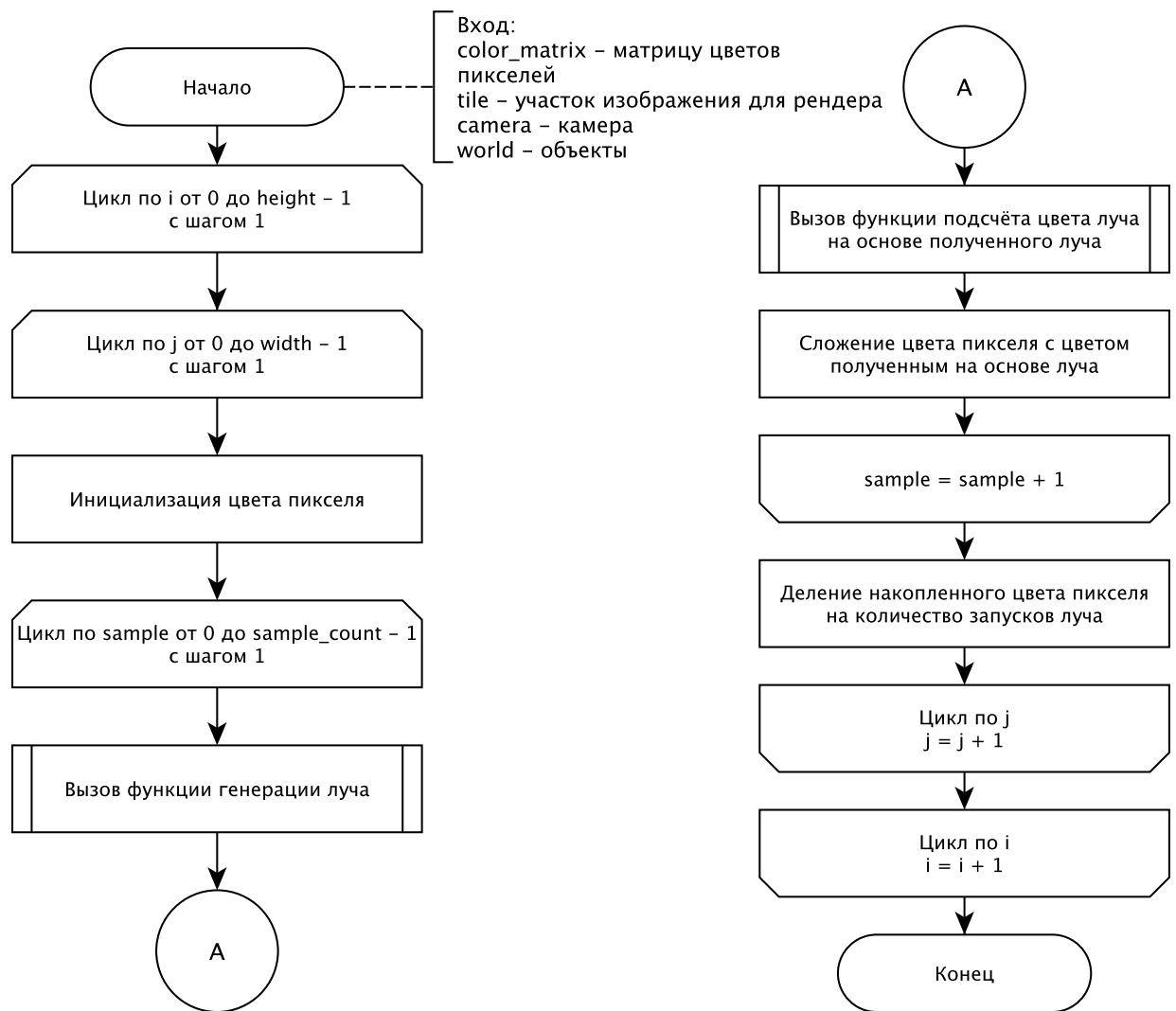


Рисунок 2.2 — Схема реализации алгоритма генерации участка изображения

На рисунках 2.3 и 2.4 изображена схема реализации главного потока параллельного алгоритма.

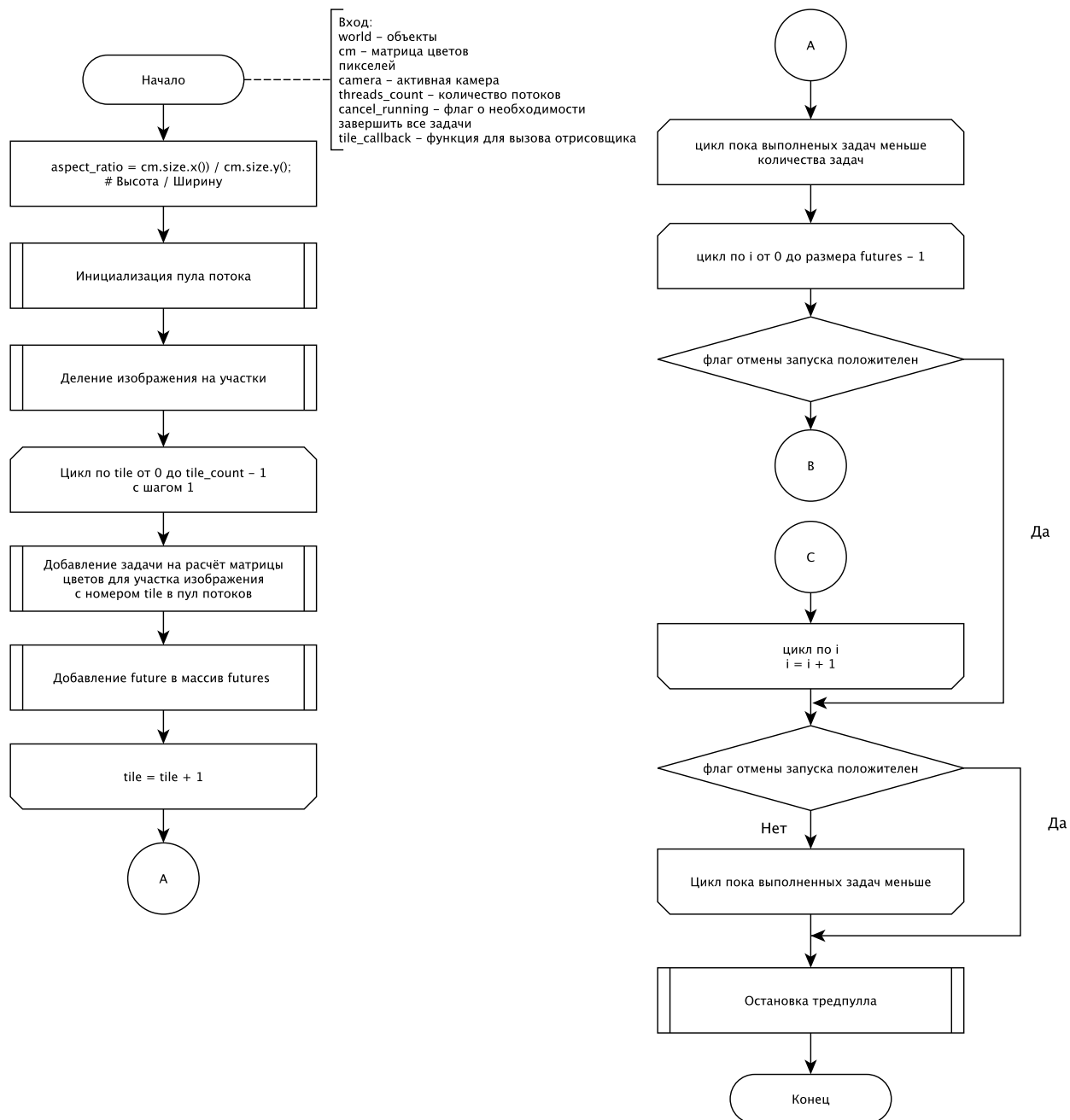


Рисунок 2.3 — Схема реализации главного потока параллельного алгоритма – ч. 1

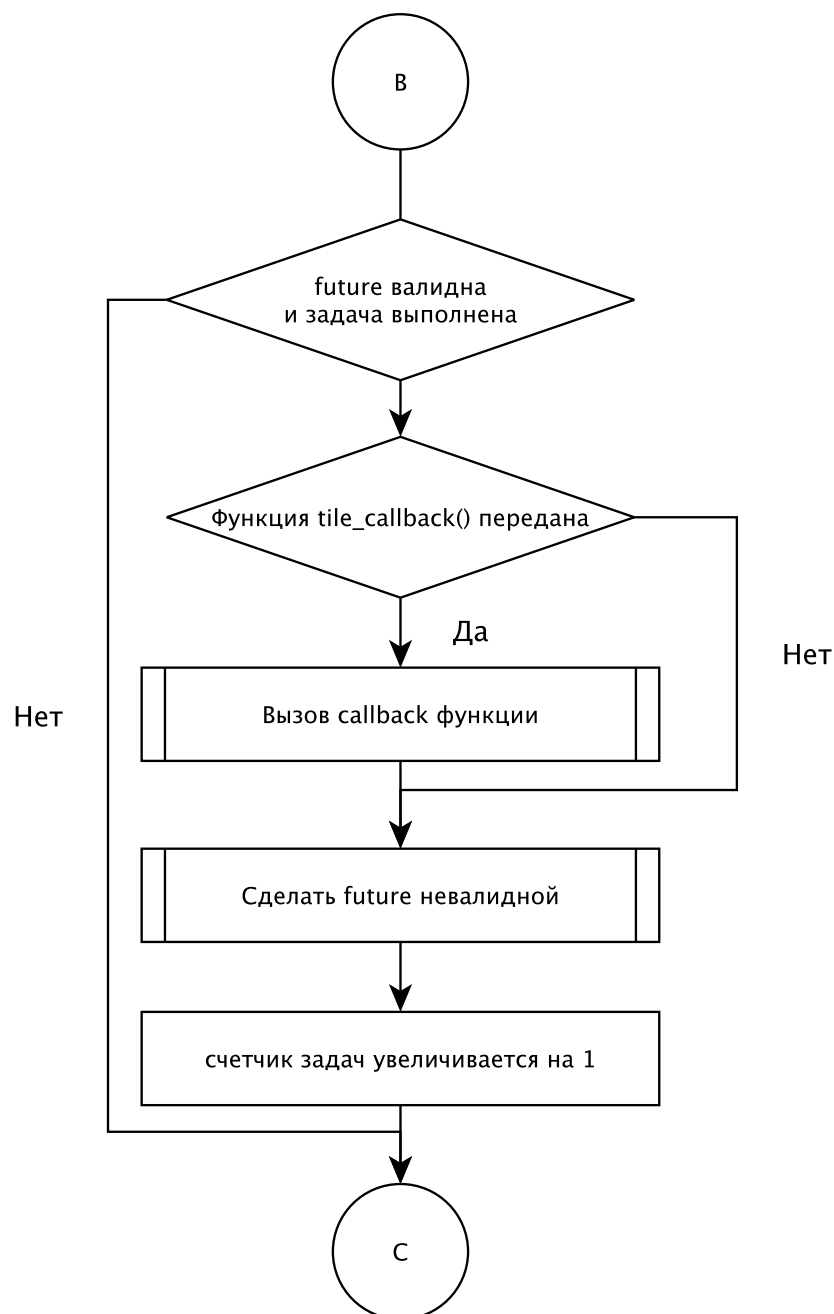


Рисунок 2.4 — Схема реализации главного потока параллельного алгоритма – ч. 2

2.2 Разработка средства для параллельного выполнения программы

Для реализации моего варианта была разработана собственная реализация пула потоков. Поскольку изображение разбивается на большое количество небольших областей, создание отдельного потока для каждой задачи привело бы к значительным накладным расходам: выделение стека, инициализация в ядре ОС и последующее переключение контекста. Чтобы избежать этих издержек, все задачи помещаются в общую очередь, а фиксированное число рабочих потоков последовательно извлекают и выполняют их по мере освобождения. Такой подход минимизирует

затраты на создание потоков и обеспечивает эффективную утилизацию ресурсов.

На рисунке 2.5 изображена схема реализации рабочего потока пула потоков.

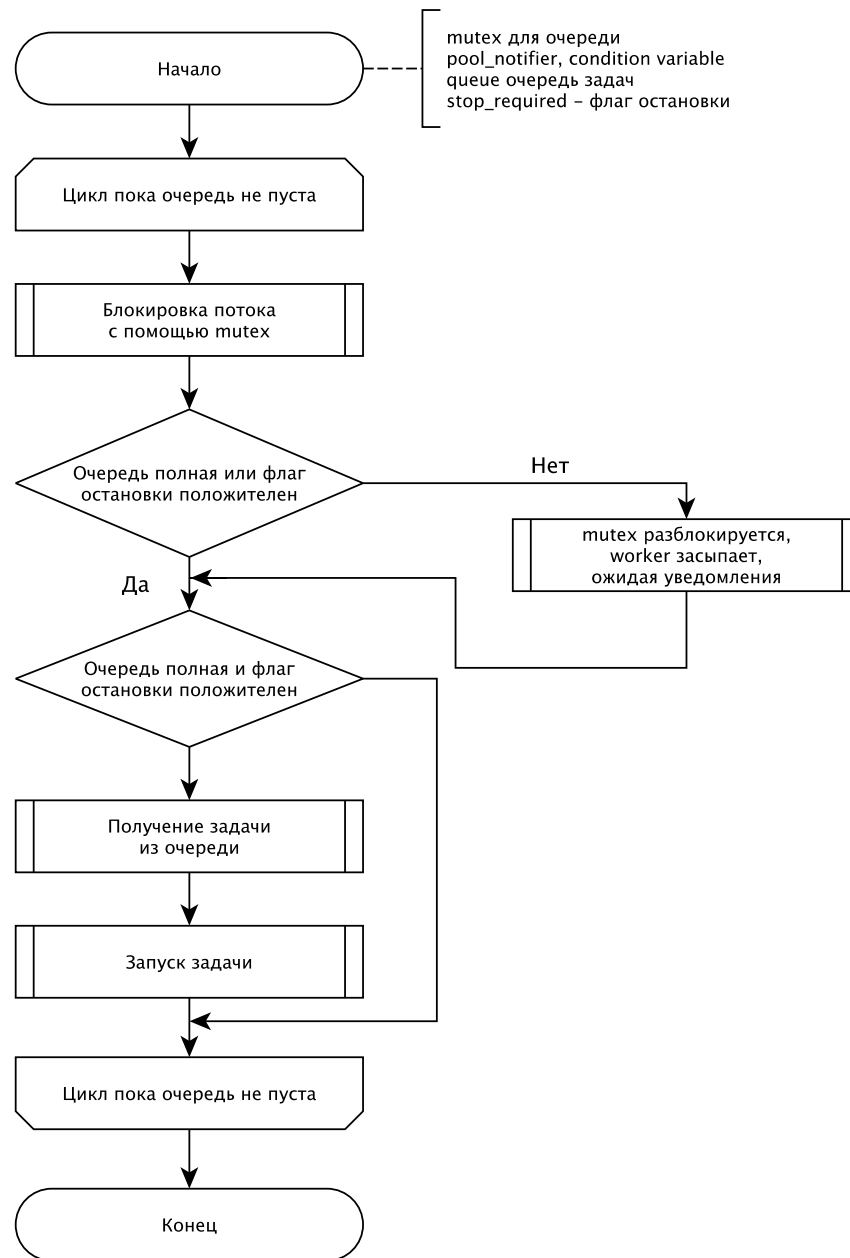


Рисунок 2.5 — Схема реализации рабочего потока пула потоков

2.3 Генерация изображения по областям

Для улучшения пользовательского опыта было принято решение открывать интерфейс сразу, а не после генерации всего изображения. При создании изображения, области выводятся в интерфейс сразу после окончания их генерации. Чтобы не блокировать основной поток с интерфейсом, необходимо разделить программу на 2 потока:

- поток для работы с графическим фреймворком;
- поток для генерации изображения.

2.4 Типы данных

В качестве примитивов синхронизации были выбраны *std :: mutex* [2, с. 1784–1791] и *std :: condition_variable* [2, с. 1583–1690]. Для возможности получения результата задачи использовался *std :: future* [2, с. 1595–1602].

Данные о матрице цветов пикселей хранятся в классе *ColorMatrix*, представляющий собой *std::vector*. Количество потоков задаётся типом данных *size_t*. Флаг отмены задаётся типом *bool*, а callback функция с помощью класса *std :: function < void >*

Для запуска генерации в отдельном потоке и обхода блокировки графического потока, использовался метод *run()* из класса *QtConcurrent* [3].

Вывод

В данной части были представлены схемы последовательного и параллельного алгоритма, реализации функции генерации участка изображения и реализации рабочего потока.

3 Технологическая часть

В данной части будут рассмотрены средства реализации программного обеспечения, предоставлены листинги и приведены тестовые данные

3.1 Средства реализации

В лабораторных работах необходимо использовать язык со статической типизацией, который поддерживает создание нативных потоков. По этой причине был выбран язык C++ [2]. Разработка проводилась в среде программирования Visual Studio Code. Для сборки использована утилита CMake. Для визуализации данных, полученных в ходе исследования использовался язык Python [4] за счёт обилия библиотек для анализа данных. Мною были использованы pandas [5] и matplotlib [6]. Для запуска тестов и последующего сохранения результатов также использовался язык Python.

3.1.1 Реализация пула потоков

Для эффективной работы с множеством задач был реализован пул потоков. Он предоставляет механизм для асинхронного выполнения задач с использованием фиксированного количества рабочих потоков.

При создании объекта класса запускается указанное число потоков, каждый из которых ожидает поступления задач через очередь. Исходный код рабочего потока указан в листинге 3.1. Задачи добавляются методом *add_task*, приведённом в листинге 3.2. Он принимает произвольную вызываемую сущность и её аргументы, упаковывает их в *std::packaged_task* помещает в очередь и возвращает *std::future* [2, с. 1595–1602] для получения результата. Внутренняя синхронизация обеспечивается с помощью *std::mutex* и переменной условия *std::condition_variable* [2, с. 1583–1690], что позволяет рабочим потокам эффективно ожидать появления новых задач или сигнала завершения. При вызове деструктора или метода *stop()* устанавливается флаг остановки, все ожидающие потоки уведомляются, завершают обработку и соединяются с основным потоком.

```
void worker() {
    while (true) {
        std::packaged_task<void()> task;{
            std::unique_lock l(_mutex_queue);
            _pool_notifier.wait(l, [this]() { return !_task_queue.empty() ||
                _stop_required; });
            if (_task_queue.empty() && _stop_required) {return;}
            task = std::move(_task_queue.front());
            _task_queue.pop();}
        task();}}
```

Листинг 3.1 — Реализация рабочего потока

```
template <typename Func, typename... Args>
std::future<std::invoke_result_t<Func, Args...>> add_task(Func &&func,
Args &&...args) {
    using ret = std::invoke_result_t<Func, Args...>;
    std::packaged_task<ret()> task(
    [func = std::forward<Func>(func), args_tuple = std::make_tuple(std::
        forward<Args>(args)...)] {
        return std::apply([&](auto &&...captured_args) -> ret {
            return std::invoke(func, std::forward<decltype(captured_args)>(
                captured_args)...);},
            std::move(args_tuple));});

    std::future<ret> fut = task.get_future();
    {
        std::lock_guard l(_mutex_queue);
        if (_stop_required)
            throw std::runtime_error("ThreadPool is stopping or stopped");
        _task_queue.emplace([task = std::move(task)]() mutable {task();});
    }
    _pool_notifier.notify_one();
    return fut;}
```

Листинг 3.2 — Реализация рабочего потока

Пример использования собственной реализации пула потоков приведён на листинге 3.3.

```
ThreadPool pool(thread_count);
for (auto tile : tiles)
{
    auto fut = pool.add_task([&color_matrix, tile, &render_task, &world
        ]()
    { render_task->execute(color_matrix, *tile, world); });
    futures.push_back(std::move(fut));
}
pool.stop();
```

Листинг 3.3 — Реализация рабочего потока

3.2 Управление нативными потоками

Класс `std::thread` [2, с. 1784–1791], входящий в стандартную библиотеку C++, предоставляет высокоуровневый интерфейс для работы с потоками. Он реализован поверх натив-

ных POSIX-потоков, что обеспечивает эффективное взаимодействие с операционной системой. Создание нового потока осуществляется путём вызова конструктора класса `std::thread` с передачей ему вызываемого объекта, которая начинает выполняться в отдельном потоке. Для синхронизации выполнения основного и дочернего потоков используется метод `join()`, который блокирует вызывающий поток до завершения целевого. Эти возможности полностью покрывают требования, поставленные в лабораторной работе, позволяя реализовать необходимую логику параллельного выполнения. Пример работы с классом `std::thread` показан в листинге 3.4. вызываемого.

```
#include <thread>

void foo() {}
void bar(int x) {}

int main() {
    std::thread first(foo);           // create pool 1
    std::thread second(bar,0);        // create pool 2

    first.join();
    second.join();
}
```

Листинг 3.4 — Пример работы с библиотекой `std::thread`

3.3 Реализация замеров времени выполнения реализации алгоритмов

Для измерения времени выполнения реализации алгоритмов использовалась библиотека *chrono* [2, с. 1250–1260], входящая в стандартную. Из этого класса для получения текущего времени использовались функции `chrono::high_resolution_clock::now()` [2, с. 1258] для получения реального времени и `chrono::duration_cast<>()` [2, с. 1250] для приведения времени к необходимому формату. Для проведения замеров была отключена оптимизация компилятора флагом `-o0` [7].

3.4 Реализация генерации изображения по областям

Для того, чтобы генерация изображения не блокировала поток графического приложения, было принято решение разделить программу на 2 потока. Разделение происходило с помощью функции `run()` из класса *QtConcurrent* [3]. Для получения результата функции запуска генерации изображения и вызова сигнала об окончании генерации использовался класс *QFutureWatcher* [8]. Метод `invokeMethod()` класса *QMetaObject* нужен для связи фонового и основного потока. Лямбда функция с вызовом `invokeMethod()` используется в качестве

функции обратного вызова, которая вызывается каждый раз при завершении генерации области. Код запуска функции генерации кадра из интерфейса приведен на листинге 3.5

```
_render_color_matrix = std::make_shared<ColorMatrix>(size.height(),
    size.width());
_render_pixmap = std::make_shared<QPixmap>(size);
_drawer = std::make_shared<Drawer>(*_render_pixmap);
_render_pixmap->fill(Qt::black);

cancel_running = false;
_futureWatcher.setFuture(QtConcurrent::run([this, size]() {
    _scene->draw(size, *_render_color_matrix, cancel_running, [this]() {
        QMetaObject::invokeMethod(this, "tile_render_finished_slot",
            Qt::QueuedConnection);
    });
}));
```

Листинг 3.5 — Запуск функции генерации кадра из интерфейса

3.5 Реализация алгоритмов

Основной цикл алгоритма, подвергаемый распараллеливанию, обращается к общему ресурсу — матрице, однако каждый поток обрабатывает строго определённую часть изображения, вследствие чего их рабочие области в памяти не пересекаются. Это исключает необходимость применения примитивов синхронизации, таких как мьютексы или семафоры. По той же причине отсутствует потребность в локальном вычислении промежуточных данных внутри каждого потока с последующим их объединением.

Исходный код реализации последовательного алгоритма представлен в листинге 3.6. Исходный код реализации параллельного алгоритма представлен в листинге 3.7. Исходный код реализации алгоритма генерации части изображения представлен в листинге 3.8.

```
void Render::render_seq(const Hittable &world, ColorMatrix &
    color_matrix)
{
    _size = color_matrix.size();
    initialize();
    auto render_task = std::make_shared<RenderTask>(*this);
    render_task->execute(color_matrix, Tile(0, 0, _size.x(), _size.y()),
        world);
}
```

Листинг 3.6 — Исходный код последовательного алгоритма генерации кадра

```
void Render::render(const Hittable &world, ColorMatrix &color_matrix,
```

```

volatile bool &cancel_running, size_t thread_count,
std::function<void()> tile_callback)
{
    _size = color_matrix.size();
    initialize();

    ThreadPool pool(thread_count);

    auto tile_creator = std::make_shared<TileCreator>(_size.x(), _size.y
        ());
    auto tiles = tile_creator->create(48);

    auto render_task = std::make_shared<RenderTask>(*this);
    std::vector<std::future<void>> futures;
    for (auto tile : tiles)
    {
        auto fut = pool.add_task([&color_matrix, tile, &render_task, &world
            ]()
            { render_task->execute(color_matrix, *tile, world); });
        futures.push_back(std::move(fut));
    }

    size_t completed = 0;
    while (completed < tiles.size())
    {
        for (size_t i = 0; i < futures.size(); ++i)
        {
            if (cancel_running)
                break;
            if (futures[i].valid() && futures[i].wait_for(std::chrono::
                milliseconds(
                0)) == std::future_status::ready)
            {
                if (tile_callback)
                    tile_callback();
                futures[i] = std::future<void>{};
                completed++;
            }
        }

        if (cancel_running)
            break;
    }
}

```

```
pool.stop();}
```

Листинг 3.7 — Исходный код параллельного алгоритма генерации кадра

```
void RenderTask::execute(ColorMatrix &color_matrix, const Tile &tile,
    const Hittable &world) {
    for (int i = tile.i; i < tile.end_i; i++) {
        for (int j = tile.j; j < tile.end_j; j++) {
            Color pixel_color(0, 0, 0);
            for (int sample = 0; sample < _camera.samples_per_pixel; sample
                ++){
                Ray ray = _camera.get_ray(j, i);
                pixel_color += _camera.ray_color(ray, _camera.max_depth, world)
                    ;}
            color_matrix.at(i, j) = _camera.pixel_samples_scale * pixel_color
                ;}}}
```

Листинг 3.8 — Исходный код реализации алгоритма генерации части изображения представлен в листинге

3.6 Тестирование ПО

Для тестирования реализованного ПО были выделены тестовые случаи, указанные в таблице 3.1. Все тесты пройдены успешно.

Таблица 3.1 — Функциональные тесты

Вход алгоритма	Ожидаемый результат	Фактический результат
Объект класса рисования не установлен	Painter not set!	Painter not set!
Массив камер пуст	Camera not set!	Camera not set!
Массив объектов сцены пуст	Кадр неба	Кадр неба

Вывод

В данной части описаны инструменты, использованные для реализации алгоритмов и пула потоков, а также рассмотрен подход к управлению нативными потоками. Приведены исходные коды параллельной и последовательной версий алгоритма, а также представлены результаты функционального тестирования разработанных решений.

4 Исследовательская часть

В данной части будет проведён сравнительный анализ последовательного и параллельного алгоритмов генерации кадра с использованием трассировки лучей.

4.1 Технические характеристики

Исследования проводились на устройстве со следующими характеристиками [10]:

- операционная система MacOS Tahoe 26.0.1 (25A362);
- процессор Apple M3, 8 ядер — 4 производительных и 4 энергоэффективных;
- оперативная память 16 Гигабайт, LPDDR5;

Во время замеров времени ноутбук был подключён к зарядному устройству. Никакие пользовательские задачи в фоне не выполнялись. Оптимизация компилятора выключена флагом -o0 [7].

4.2 Временя выполнения последовательной и параллельной реализации алгоритма

Была исследована зависимость времени выполнения последовательной и параллельной реализации алгоритма от количества объектов на сцене. Для каждого значения варьируемого параметра измерение проводилось 100 раз, полученные значения времени суммировались, после чего усреднялись. Результаты исследования приведены в таблице 4.1

Таблица 4.1 — Сравнение времени выполнения реализаций последовательного и параллельного алгоритмов

Количество объектов	Время выполнения последовательной реализации, мс	Время выполнения параллельной реализации, мс
2	172	180
4	197	201
6	217	221
8	235	239
10	252	261
12	268	272
14	289	294
16	310	315
18	326	331

На рисунке 4.1 изображены графики зависимости времени выполнения алгоритма от количества объектов, попадающих в кадр.

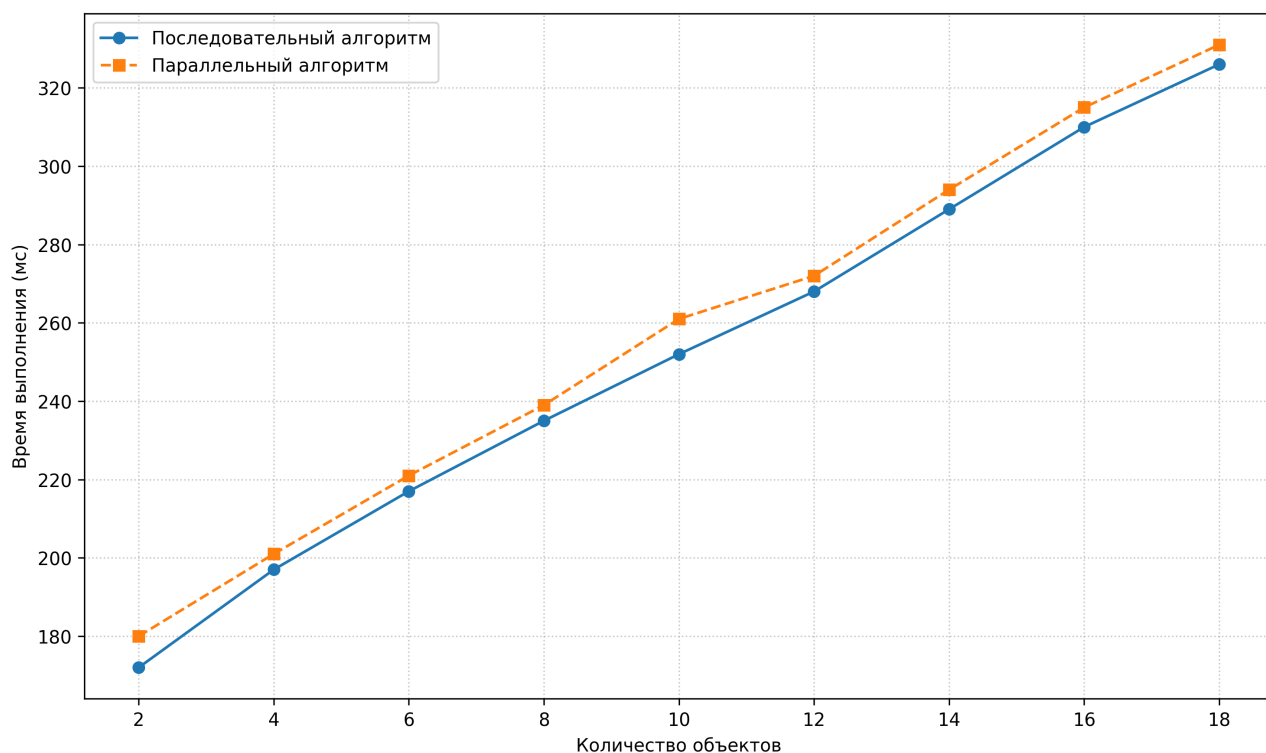


Рисунок 4.1 — Графики зависимости времени выполнения реализации алгоритма от количества объектов

При использовании одного рабочего потока последовательный алгоритм в среднем превосходит параллельный по производительности на 10%. Данное различие обусловлено более простой логикой последовательной реализации, в то время как параллельная версия несёт накладные расходы, связанные с организацией и управлением пулом потоков.

4.3 Исследование зависимости времени выполнения реализации алгоритма от числа потоков

Была исследована зависимость времени выполнения параллельного алгоритма генерации кадра от количества рабочих потоков и количества объектов, попадающих в кадр. Результаты исследования приведены в таблице 4.2

Таблица 4.2 — Время выполнения реализации параллельного алгоритма в зависимости от количества потоков

Количество потоков	Количество видимых объектов							
	1	2	3	5	10	15	20	25
	Время выполнения, мс							
1	85	103	113	138	202	252	304	386
2	47	52	59	71	100	144	178	211
4	27	29	32	40	56	73	89	105
8	21	22	25	30	40	63	82	98
16	23	24	27	35	45	52	66	95
24	24	25	29	32	40	50	60	97
32	26	27	28	32	42	49	66	96
64	26	28	27	31	44	51	62	98
128	27	29	30	34	43	52	61	99

На рисунке 4.2 приведены результаты замеров процессорного времени выполнения реализации параллельного алгоритма для разного числа потоков.

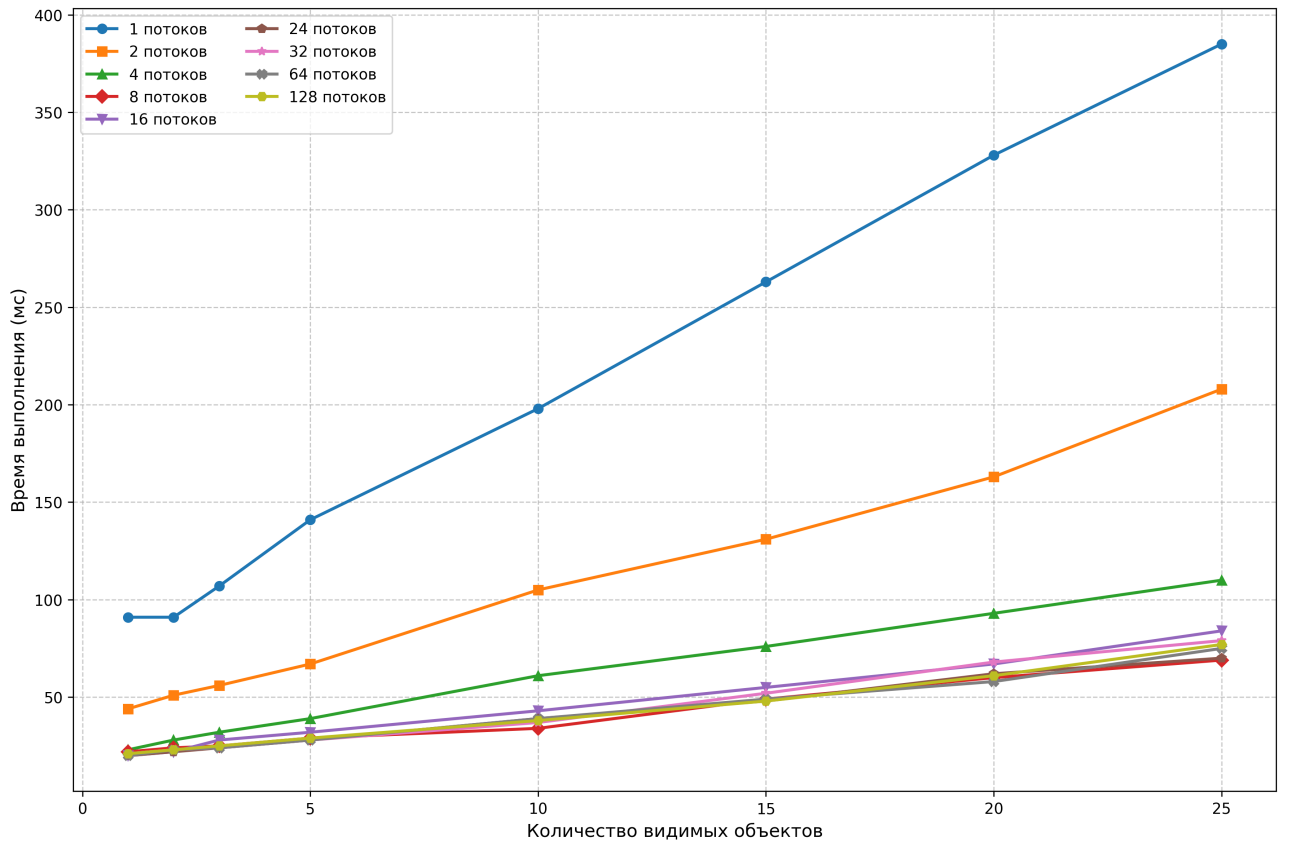


Рисунок 4.2 — Время выполнения реализации параллельного алгоритма в зависимости от количества потоков

Вывод

В данной части представлены технические характеристики используемой вычислительной системы и результаты исследования, проведённого на ней. По итогам анализа полученных данных сделаны следующие выводы.

- Время выполнения последовательной реализации алгоритма и параллельной реализации с одним рабочим потоком практически совпадает при любом количестве видимых объектов. Это свидетельствует о том, что создание и запуск одного потока сопряжены с минимальными накладными расходами.
- С ростом числа потоков время выполнения уменьшается. Это заметно до 8 потоков, после чего время выполнения начинает расти. Это связано с тем, что при превышении аппаратных возможностей процессора накладные расходы на управление потоками (создание, переключение контекста) начинают преобладать над выгодой от распараллеливания.
- Наилучшее время выполнения параллельного алгоритма достигается при использовании 8 потоков. Используемый процессор обладает 8 физическими и 8 логическими ядрами.
- При увеличении количества видимых объектов время выполнения как последовательной, так и параллельной реализации растёт линейно.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы № 4 была достигнута цель — разработан и проведён сравнительный анализ последовательного и параллельного алгоритма трассировки лучей. Были выполнены все задачи:

- 1) описан последовательный алгоритм решения задачи;
- 2) разработан параллельную версию алгоритма;
- 3) реализован обе версии алгоритма;
- 4) выполнен сравнительный анализ зависимостей времени решения задач от размерности входа для реализации последовательного алгоритма и для реализации модифицированного алгоритма;
- 5) выполнен сравнительный анализ зависимостей времени решения задач от размерности входа для реализации модифицированного алгоритма при $k \in \{1, 2, 4, 8, \dots, 8 \cdot q\}$, где q — количество логических ядер процессора;
- 6) сформулирована рекомендация по выбору k для решения поставленной задачи.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. raytrace_ebook.pdf [Электронный ресурс]. URL: http://cosinekitty.com/raytrace/raytrace_ebook.pdf (дата обращения: 14.11.2025).
2. *Standard C++* [Электронный ресурс]. URL: <https://isocpp.org/files/papers/N4860.pdf> (дата обращения: 17.11.2025).
3. Qt Concurrent Run [Электронный ресурс]. URL: <https://doc.qt.io/qt-6/qtconcurrentrun.html> (дата обращения: 14.11.2025).
4. *Python* [Электронный ресурс]. URL: <https://www.python.org/> (дата обращения: 14.11.2025).
5. *Pandas* [Электронный ресурс]. URL: <https://pandas.pydata.org/> (дата обращения: 17.11.2025).
6. *Matplotlib — Visualization with Python* [Электронный ресурс]. URL: <https://matplotlib.org/> (Дата обращения: 17.11.2025).
7. *Clang command line argument reference* [Электронный ресурс]. URL: <https://clang.llvm.org/docs/ClangCommandLineReference.html#optimization-level> (Дата обращения: 17.11.2025).
8. QFutureWatcher Class [Электронный ресурс]. URL: <https://doc.qt.io/qt-6/qfuturewatcher.html> (дата обращения: 17.11.2025).
9. QMetaObject Struct [Электронный ресурс]. URL: <https://doc.qt.io/qt-6/qmetaobject.html> (дата обращения: 17.11.2025).
10. *Macbook air m3* [Электронный ресурс]. URL: <https://support.apple.com/en-us/118551> (Дата обращения: 17.11.2025).